

```

// server.js

const express = require('express');
const jwt = require('jsonwebtoken');
require('dotenv').config();

const app = express();
app.use(express.json());

// Sample user data (in real apps, use a database)
const users = [
  { id: 1, username: 'user1', password: 'pass1' },
  { id: 2, username: 'user2', password: 'pass2' },
  { id: 3, username: 'user3', password: 'pass3' }
];

// Secret key for JWT signing
const ACCESS_TOKEN_SECRET = process.env.ACCESS_TOKEN_SECRET ||
  'your-secret-key';

// Middleware to authenticate and authorize requests using JWT
function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Bearer <token>

  if (!token) return res.status(401).json({ message: 'Access token
missing' });

  jwt.verify(token, ACCESS_TOKEN_SECRET, (err, user) => {
    if (err) return res.status(403).json({ message: 'Invalid or expired
token' });
    req.user = user; // Attach user info to request
    next();
  });
}

// Login route - issues JWT on valid credentials
app.post('/login', (req, res) => {
  const { username, password } = req.body;

```

```
// Find user
const user = users.find(u => u.username === username && u.password ===
password);
if (!user) return res.status(401).json({ message: 'Invalid credentials'
});

// Create JWT payload
const payload = { id: user.id, username: user.username };

// Sign JWT with expiration (e.g., 1 hour)
const accessToken = jwt.sign(payload, ACCESS_TOKEN_SECRET, { expiresIn:
'1h' });

res.json({ accessToken });
});

// Protected endpoint - only accessible with valid JWT
app.get('/dashboard', authenticateToken, (req, res) => {
  res.json({
    message: `Welcome to your dashboard, ${req.user.username}!`,
    userData: req.user
  });
});

// Start server
const PORT = 3000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Execution Process

1. Set up project:

```
mkdir jwt-auth  
cd jwt-auth  
npm init -y  
npm install express jsonwebtoken dotenv
```

2. Create **.env** file:

```
ACCESS_TOKEN_SECRET=your-very-secure-secret
```

3. Save the code above into [server.js](#).

4. Run the server:

```
node server.js
```

5. Use Postman (or any HTTP client) to test:

Login to get token:

- **POST** `http://localhost:3000/login`

- **Body (JSON):**

```
{  
  "username": "user1",  
  "password": "pass1"  
}
```

Response:

```
{  
  "accessToken": "<your-jwt-token>"  
}
```

Access protected endpoint:

- **GET** `http://localhost:3000/dashboard`
- **Headers:**

Authorization: Bearer <your-jwt-token>

Response (if token valid):

```
{  
  "message": "Welcome to your dashboard, user1!",  
  "userData": {  
    "id": 1,  
    "username": "user1",  
    "iat": 1696500000,  
    "exp": 1696503600  
  }  
}
```